

QQ 浏览器前端一面面经

QQ 浏览器前端一面面经

1. 自我介绍
2. 问项目做了什么，有什么难点
3. 详细说下 React Diff 算法
4. Vue 中父组件给子组件传数据
5. 如果是跨很多层级的话怎么办（续）
6. localStorage 和 sessionStorage 什么时候会消失，怎么样清除
7. 怎么使用原型的方法
8. 怎么样去创建一个原型（差不多）
9. 用类去创建适用于所有浏览器吗？
10. 如果要去适配低版本浏览器怎么做？
11. webpack 有哪些了解？
12. 为什么需要 webpack？
13. 说一下闭包
14. 为什么闭包的变量不会消失
15. 垃圾回收机制怎么实现的
16. 常用的数据结构（答了链表和数组）
17. 链表数组的区别（续）
18. 两者使用场景（）
19. 哈希表的原理
20. HTTP 包含哪些内容
21. HTTPS 和 HTTP 区别
22. SSL 证书能够确保来源的（大概意思是 SSL 证书的认证机构）

23.TCP 和 HTTP 的关系

24.UDP 和 TCP 区别

25.TCP 怎么确保数据的可靠性

26. 为什么需要三次握手

27. 三次握手状态有哪些改变

28. 说说 cookie 和 session

29. 随便写一个 sessionId 会不会窃取到用户信息

30. 场景题：假如现在有 10 亿条十位数 QQ 号数据，你需要从中找出其中一条，怎么做

参考答案

3.React Diff 算法

- React 的 Diff 算法，也称为虚拟 DOM Diffing 算法，是 React 用于高效更新 UI 的核心机制。当 React 组件的状态或属性发生变化时，它会生成一个新的虚拟 DOM 树，并与旧的虚拟 DOM 树进行比较。通过比较两个虚拟 DOM 树的差异，React 可以确定需要更新的最小集合，并仅更新这些部分，而不是重新渲染整个页面。这种机制大大提高了 React 应用的性能和响应速度。

4.Vue 中父组件给子组件传数据

- 在 Vue 中，父组件可以通过属性（props）将数据传递给子组件。在父组件的模板中，可以使用 v-bind 指令（或简写为 :）将需要传递的数据绑定到子组件的属性上。子组件则通过 props 选项来接收这些数据，并在组件内部使用它们。

5. 跨很多层级传递数据

- 如果需要在 Vue 中跨多个层级传递数据，可以使用 Vuex（Vue 的状态管理模式）或事件总线（Event Bus）等机制。Vuex 是一个专为 Vue.js 应用程序开发的状态管理模式库，它采用集中式存储管理应用的所有组件的状态，

localStorage 和 sessionStorage 是 Web 存储 API 的一部分，它们用于在客户端存储数据，但有着不同的生命周期和清除方式。

6.localStorage

1. 消失时机：

localStorage 中的数据没有过期时间，数据会在浏览器关闭后继续保留，直到用户手动清除，或通过 JavaScript 代码调用 localStorage.clear() 方法删除。

当用户主动清除浏览器缓存时，localStorage 中的数据也会被清除。

2. 清除方式：

使用 JavaScript 代码：localStorage.clear() 方法可以清除所有存储的数据。

通过 localStorage.removeItem(key) 方法可以删除特定键的数据。

手动清除：用户可以在浏览器设置中清除网站数据，包括 localStorage。

sessionStorage

1. 消失时机：

sessionStorage 中的数据仅在页面会话期间有效。当用户关闭所有与网站相关的窗口或标签页时，sessionStorage 会被清除。

当用户主动清除浏览器缓存和数据时，sessionStorage 也会被清除。

如果页面使用了 "sessionStorage.clear()" 方法来清除 sessionStorage 中的数据，那么所有的会话数据将会被立即删除。

2. 清除方式：

使用 JavaScript 代码：sessionStorage.clear() 方法可以清除所有存储的数据。

通过 sessionStorage.removeItem(key) 方法可以删除特定键的数据。

页面会话结束：当用户关闭所有相关窗口或标签页时，sessionStorage 会自动清除。

总结来说，localStorage 和 sessionStorage 的消失时机和清除方式有所不同。

localStorage 适合用于长期存储数据，如用户偏好设置、主题选择等；而 sessionStorage 则适用于临时会话，如表单输入、页面间传递的临时数据等。在需要时，可以通过 JavaScript 代码或手动方式清除这些数据。

7. 怎么使用原型的方法：

在 JavaScript 中，原型（prototype）是对象的一个内置属性，它允许对象共享方法和属性。要使用原型的方法，可以通过对象的原型链来访问这些方法。具体来说，可以定义一个构造函数，并在其原型上添加方法，然后通过该构造函数创建的对象就可以访问这些方法了。例如：

javascript

```
1 function Person(name) {  
2   this.name = name;  
3 }  
4  
5 Person.prototype.sayHello = function() {  
6   console.log("Hello, my name is " + this.name);  
7 };  
8  
9 const person1 = new Person("Alice");  
10 person1.sayHello(); // 输出: Hello, my name is Alice
```

8. 怎么样去创建一个原型：

在 JavaScript 中，创建原型的方法通常与构造函数一起使用。通过为构造函数添加一个 `prototype` 属性，并在该属性上定义方法，就可以为通过该构造函数创建的所有对象提供共享的方法和属性。例如上面的 `Person` 构造函数和它的 `prototype` 方法 `sayHello`。

9. 用类去创建适用于所有浏览器吗：

使用 ES6 引入的 `class` 语法来创建对象在现代浏览器中通常是受支持的，包括 Chrome、Firefox、Safari、Edge 等。然而，一些非常旧的浏览器（如 IE 10 及更早版本）可能不支持 `class` 语法。为了确保在所有浏览器中都能正常工作，可以使用 Babel 等转译器将 ES6 代码转换为 ES5 代码，或者简单地使用传统的构造函数和原型来创建对象。

10. 如果要去适配低版本浏览器怎么做：

要适配低版本浏览器，可以采取以下措施：

使用 Polyfill：Polyfill 是一种浏览器技术的补丁，用于在旧版浏览器中提供那些原本不受支持的现代功能。

使用转译器：如 Babel 等转译器可以将 ES6+ 的代码转换为 ES5 代码，以确保在旧版浏览器中运行。

条件加载：根据浏览器的版本和特性来决定加载哪些脚本和样式。

渐进增强与优雅降级：确保网站在旧版浏览器中也能基本使用（优雅降级），同时在新版浏览器中提供更好的用户体验（渐进增强）。

11.webpack 有哪些了解：

Webpack 是一个现代 JavaScript 应用程序的静态模块打包器（module bundler）。它将应用程序的所有依赖项（包括 JavaScript 文件、CSS 文件、图片等）打包成一个或多个 bundle，这些 bundle 可以在浏览器中快速加载。Webpack 支持代码分割、懒加载、热模块替换等高级功能，并提供了强大的插件系统来扩展其功能。

12. 为什么需要 webpack：

需要 Webpack 的原因主要有以下几点：

模块化：Webpack 允许开发者使用模块化的方式来组织代码，使得代码更加清晰、易于维护。

打包优化：Webpack 可以将应用程序的所有依赖项打包成一个或多个 bundle，从而减少 HTTP 请求的数量和大小，提高加载速度。

代码分割与懒加载：Webpack 支持代码分割和懒加载，使得只有在需要时才加载某些代码块，从而进一步优化加载性能。

插件系统：Webpack 提供了强大的插件系统，允许开发者根据自己的需求来扩展 Webpack 的功能。

13. 说一下闭包：

闭包是指一个函数能够记住并访问它的词法作用域（即定义它时的作用域）中的变量，即使这个函数在词法作用域之外执行。简单来说，闭包就是函数和其相关引用环境组合而成的实体。

14. 为什么闭包的变量不会消失：

闭包中的变量不会消失的原因在于 JavaScript 的作用域链和垃圾回收机制。当函数被创建时，它会记住并保存自己的词法作用域（即定义它的环境）。即使这个函数被调用到了其他作用域中，它仍然可以通过作用域链访问到原来的词法作用域中的变量。此外，由于垃圾回收机制在判断一个对象是否可以被回收时，会看该对象是否还有被引用的可能。而闭包中的函数作为对原词法作用域中变量的引用，阻止了这些变量被垃圾回收机制回收。

15. 垃圾回收机制怎么实现的：

JavaScript 的垃圾回收机制是自动管理内存分配和释放的过程。它主要通过两种策略实现：标记 - 清除（Mark-and-Sweep）和引用计数（Reference Counting）。现代 JavaScript 引擎如 V8（Chrome 和 Node.js 中使用）和 SpiderMonkey（Firefox 中使用）主要采用标记 - 清除算法，并在此基础上做了优化，如增量标记、分代回收等。

标记 - 清除算法：从根对象（全局变量、当前执行上下文中的变量等）出发，遍历所有可达的对象，并将它们标记为活动的。然后遍历堆内存，未被标记的对象被认为是垃圾，可以被回收。

引用计数算法：每个对象都有一个引用计数，每当有新的引用指向它时计数加 1，引用消失时减 1。当计数降为 0 时，对象被视为垃圾可回收。不过，现代 JavaScript 引擎通常不单独使用引用计数算法，而是将其作为标记 - 清除算法的一种辅助手段。

16. 常用的数据结构（答了链表和数组）：

链表和数组是 JavaScript 中常用的数据结构。

数组：数组是一种固定大小（但可动态调整，需要时创建新数组并复制元素）、连续存储的数据结构。它允许通过索引快速访问元素，但插入和删除操作可能涉及移动其他元素。

链表：链表是一种非连续、动态分配的数据结构。它由节点组成，每个节点包含数据元素和指向下一个节点的指针。链表允许动态调整大小，插入和删除操作只需要更改相应节点的指针，但访问元素需要从头节点开始逐个遍历。

17. 链表数组的区别（续）：

链表和数组的主要区别体现在以下几个方面：

存储方式：数组是连续存储的，而链表是非连续存储的。

访问速度：数组可以通过索引快速访问元素，时间复杂度为 $O(1)$ ；而链表需要从头节点开始逐个遍历，时间复杂度最坏为 $O(n)$ 。

插入和删除操作：数组在插入和删除元素时可能需要移动其他元素，时间复杂度为 $O(n)$ ；而链表只需要更改相应节点的指针，时间复杂度为 $O(1)$ （在已知插入或删除位置的情况下）。

内存开销：数组不需要额外的指针来连接元素，内存使用效率较高；而链表的节点需要额外的指针来连接，因此内存使用效率较低。

18. 两者使用场景：

数组适用于需要频繁访问指定位置元素的场景，如实现矩阵、二叉树等数据结构。

链表适用于需要频繁插入和删除元素的场景，如实现队列、栈、链表等数据结构。

19. 哈希表的原理：

哈希表（Hash Table）是一种基于哈希函数的数据结构。它通过对对象元素中的关键字进行哈希算法的运算，得出一个具体的哈希值，并将该值作为元素在哈希表中的位置（即索引）。如果哈希值出现冲突（即两个不同的关键字计算出相同的哈希值），则需要通过一定的冲突解决策略（如链地址法、开放地址法等）来处理。哈希表允许在 $O(1)$ 时间复杂度内进行查找、插入和删除操作（在平均情况下）。

20. HTTP 包含哪些内容：

HTTP（超文本传输协议，HyperText Transfer Protocol）是一种用于分布式、协作式、超媒体信息系统的应用层协议。它主要包括以下几个方面的内容：

请求 - 响应模型：客户端发送请求到服务器，服务器处理请求并返回响应。

资源定位：通过 URL（统一资源定位符）来定位网络上的资源。

请求方法：HTTP 定义了一组请求方法（如 GET、POST、PUT、DELETE 等）来指示要对给定资源执行的操作。

状态码：HTTP 响应中包含状态码来指示请求的成功或失败。

头部字段：HTTP 请求和响应中都包含头部字段来传递额外的信息（如内容类型、编码方式、缓存策略等）。

消息体：HTTP 请求和响应中都可以包含消息体来传递数据（如 HTML 文档、JSON 对象等）。

HTTP 是无连接的、无状态的、媒体独立的协议，它基于 TCP/IP 通信协议来传递数据。

21.HTTPS 和 HTTP 区别

HTTPS（HyperText Transfer Protocol Secure）和 HTTP（HyperText Transfer Protocol）都是用于在 Web 上传输数据的协议，但它们之间存在显著的差异。

安全性：HTTPS 在 HTTP 的基础上加入了 SSL/TLS 加密层，因此能够确保数据在传输过程中的安全性。相比之下，HTTP 是明文传输，数据容易被窃取或篡改。

端口：HTTP 默认使用 80 端口，而 HTTPS 默认使用 443 端口。

资源消耗：由于 HTTPS 需要对数据进行加密和解密，因此相比 HTTP 会消耗更多的服务器和客户端资源。

浏览器显示：在浏览器中访问 HTTPS 网站时，地址栏会显示 “https://” 前缀，并且通常会显示一个锁形图标，表示网站是安全的。而 HTTP 网站则没有这个标识。

应用场景：HTTPS 通常用于需要传输敏感信息（如信用卡信息、登录密码等）的网站。而 HTTP 则更适用于传输非敏感信息的场景。

22.SSL 证书能够确保来源的

SSL 证书（Secure Sockets Layer Certificate）是一种数字证书，用于在 Internet 上提供安全的通信。它能够确保通信双方的身份验证和数据的安全性。关于 SSL 证书能够确保的来源，主要体现在以下几个方面：

身份验证：SSL 证书通过数字签名和加密算法来验证服务器的身份。当浏览器访问一个使用 SSL 证书的网站时，它会验证证书的有效性，从而确保该网站是由证书颁发机构（CA）认证的合法网站。

数据完整性：SSL 证书使用加密算法来确保数据在传输过程中的完整性。这意味着数据在传输过程中不会被篡改或损坏。

数据保密性：SSL 证书使用加密算法对传输的数据进行加密，从而确保数据在传输过程中不会被窃取或泄露。

因此，SSL 证书能够确保通信双方的身份和数据的安全性，从而保护用户的隐私和敏感信息。

23.TCP 和 HTTP 的关系

TCP（Transmission Control Protocol）和 HTTP（HyperText Transfer Protocol）是计算机网络中两个重要的协议，它们之间存在一定的关系。

层次关系：TCP 是传输层协议，而 HTTP 是应用层协议。TCP 负责在两台计算机之间建立可靠的数据传输通道，而 HTTP 则利用这个通道来传输超文本数据。

依赖关系：HTTP 依赖于 TCP 来实现数据的可靠传输。HTTP 在传输数据时，会先将数据分割成多个 TCP 数据包，然后利用 TCP 的可靠传输机制将这些数据包发送到接收端。接收端收到数据包后，会按照 TCP 的协议进行重组，从而恢复出原始的 HTTP 数据。

功能关系：TCP 提供了面向连接的、可靠的、字节流的服务。它确保数据在传输过程中不会丢失、不会重复、不会乱序。而 HTTP 则是一种无状态的、请求 / 响应的协议，它利用 TCP 提供的可靠传输服务来传输超文本数据。

24.UDP 和 TCP 区别

UDP（User Datagram Protocol）和 TCP（Transmission Control Protocol）都是传输层协议，但它们之间存在显著的差异。

连接性：TCP 是面向连接的协议，在传输数据之前需要先建立连接。而 UDP 则是面向无连接的协议，不需要建立连接即可直接传输数据。

可靠性：TCP 提供了可靠的传输服务，通过确认机制、重传机制等确保数据在传输过程中不会丢失、不会重复、不会乱序。而 UDP 则不提供这些可靠性保障，数据在传输过程中可能会丢失或乱序。

速度：由于 UDP 不需要建立连接和进行可靠性保障，因此它的传输速度通常比 TCP 更快。这使得 UDP 更适合用于对实时性要求较高的应用场景，如视频流媒体、在线游戏等。

应用场景：TCP 通常用于需要传输大量数据且对可靠性要求较高的应用场景，如文件传输、网页浏览等。而 UDP 则更适用于对实时性要求较高且对可靠性要求不高的应用场景。

25.TCP 怎么确保数据的可靠性

TCP（Transmission Control Protocol）通过一系列机制来确保数据的可靠性，这些机制包括：

确认机制：接收端在收到数据包后，会向发送端发送一个确认包（ACK），表示已经成功接收到该数据包。发送端在收到确认包后，才会继续发送下一个数据包。如果发送端在一定时间内没有收到确认包，就会认为该数据包丢失，并重新发送该数据包。

重传机制：当发送端检测到数据包丢失时，它会重新发送该数据包，直到接收端成功接收到为止。这种重传机制可以确保数据在传输过程中不会丢失。

排序机制：TCP 会为每个数据包分配一个序列号，接收端在收到数据包后，会根据序列号对数据进行排序，从而确保数据在传输过程中不会乱序。

流量控制：TCP 通过滑动窗口协议来实现流量控制，接收端会根据自己的接收能力来告诉发送端可以发送多少数据。这可以避免发送端发送过多的数据导致接收端处理不过来而丢失数据。

拥塞控制：TCP 还通过拥塞控制算法来避免网络拥塞。当网络出现拥塞时，TCP 会降低发送速率，从而避免数据丢失和网络崩溃。

26. 为什么需要三次握手

在 TCP/IP 协议中，三次握手（Three-Way Handshake）是一种建立可靠连接的重要机制。它需要客户端和服务端之间交换三个 TCP 报文段来完成连接的建立。这种机制的原因和必要性主要体现在以下几个方面：

确认双方通信能力：通过三次握手，客户端和服务端可以确认对方是否具备通信能力，并且准备好进行数据传输。这可以确保在数据传输过程中不会出现因通信故障而导致的数据丢失或错误。

同步双方初始序列号：在 TCP 连接中，每个传输的报文段都会包含一个序列号，用于标识该报文段在数据流中的位置。通过三次握手，客户端和服务端可以同步双方的初始序列号，从而确保在后续的数据传输中能够正确地识别和重组报文段。

防止重复连接：在网络环境中，可能会出现因网络延迟或丢包等原因而导致的重复连接请求。通过三次握手，可以确保每个连接都是唯一的，并且不会出现重复连接的情况。

27. 三次握手状态有哪些改变

在 TCP 的三次握手过程中，客户端和服务器的状态会发生一系列的变化。以下是这些状态变化的详细描述：

第一次握手（SYN）：

客户端发送一个 SYN 报文段到服务器，请求建立连接。此时，客户端进入 SYN_SEND 状态。

服务器收到 SYN 报文段后，会确认该请求，并发送一个 SYN-ACK 报文段作为回应。此时，服务器进入 SYN_RECV 状态。

第二次握手 (SYN-ACK)：

客户端收到服务器的 SYN-ACK 报文段后，会确认该回应，并发送一个 ACK 报文段作为确认。此时，客户端进入 ESTABLISHED 状态，表示连接已经建立。

第三次握手 (ACK)：

服务器收到客户端的 ACK 报文段后，也会确认该连接已经建立，并进入 ESTABLISHED 状态。此时，双方都可以开始传输数据了。

28. 说说 cookie 和 session

Cookie 和 Session 都是用于在 Web 开发中保持用户状态和数据的重要机制，但它们之间存在显著的差异。

存储位置：Cookie 是将数据存储在客户端（如浏览器）的文本文件中，而 Session 则是将数据存储在服务器端。

安全性：由于 Cookie 存储在客户端，因此相对容易被篡改或窃取。而 Session 存储在服务器端，因此相对更安全。但需要注意的是，如果 Session 的 ID 被泄露或猜测到，也可能存在安全风险。

有效期：Cookie 可以设置有效期，使其在客户端存储一段时间。而 Session 则通常是在用户关闭浏览器或服务器关闭时失效。但也可以通过设置 Session 的过期时间来控制其有效期。

作用范围：Cookie 的作用范围是整个域名下的所有页面，而 Session 则通常只作用于一个特定的用户会话（Session）。这意味着，同一个域名下的不同页面可以共享同一个 Cookie，但不同用户的 Session 则是相互独立的。

数据大小：Cookie 的数据大小通常有限制（如 4KB），而 Session 则可以存储更多的数据。但需要注意的是，如果 Session 中存储的数据过多，可能会增加服务器的负担和响应时间。

29. 随便写一个 sessionId 会不会窃取到用户信息

随便写一个 sessionId 并不会直接窃取到特定用户的信息，但这样做可能会带来安全风险或产生不可预期的行为。

原因如下：

唯一性冲突：每个 sessionID 应该是唯一的，以确保服务器能够正确识别并跟踪用户的会话。如果你随便写一个 sessionID，它可能与现有用户的 sessionID 冲突，导致服务器将你的请求误认为是另一个用户的会话。

会话劫持：虽然你随机生成的 sessionID 不会直接窃取信息，但如果这个 sessionID 恰好是某个已登录用户的有效 sessionID（尽管这种情况在现实中非常罕见，但理论上存在），那么攻击者可能会利用这个 sessionID 来劫持该用户的会话，从而访问该用户的敏感信息或执行未授权的操作。

安全性考虑：sessionID 通常用于在客户端和服务器之间建立安全的会话。如果 sessionID 的生成不够随机或不够安全，它可能会被预测或破解，从而导致会话被攻击者接管。

建议：

始终使用服务器生成的、足够随机和安全的 sessionID。

避免在客户端代码中硬编码 sessionID 或将其暴露给不受信任的第三方。

定期检查并更新服务器的会话管理策略，以确保会话的安全性。

30. 场景题：假如现在有 10 亿条十位数 QQ 号数据，你需要从中找出其中一条，怎么做

在处理如此庞大的数据集时，直接遍历整个数据集来查找特定的一条记录通常是不现实的，因为这将需要非常长的时间。以下是一些可能的解决方案：

1. 数据库索引：

如果这些数据已经存储在数据库中，并且每条记录都有一个唯一的 QQ 号作为主键或索引，那么可以使用数据库提供的索引功能来快速查找特定的 QQ 号。

数据库索引可以显著提高查询速度，因为它允许数据库系统以更快的方式定位到特定的记录。

2. 哈希表：

如果数据没有存储在数据库中，或者你需要更灵活的查找方式，可以使用哈希表来存储这些数据。

哈希表允许你在 $O(1)$ 时间复杂度内查找特定的记录（在平均情况下），但前提是哈希函数是良好的并且没有冲突。

对于 10 亿条数据，你需要确保哈希表有足够的容量和适当的哈希函数来减少冲突。

3. 分块处理：

如果数据太大而无法一次性加载到内存中，可以将其分成多个小块（或分区），然后逐个块地加载和处理。

这种方法可以减少内存使用，但可能会增加处理时间，因为你需要遍历多个块来查找特定的记录。

4. 并行处理：

使用多个处理器或线程来并行处理数据，以加快查找速度。

这需要确保数据可以被有效地分割和并行处理，并且需要处理并发访问和同步问题。

5. 外部存储：

如果数据太大而无法在本地存储和处理，可以考虑使用外部存储系统（如分布式文件系统、云存储等）来存储数据。

这些系统通常提供了高效的数据访问和检索机制，可以处理大规模的数据集。

6. 特定算法：

根据具体的应用场景和数据特点，可以使用特定的算法来优化查找过程。

例如，如果数据是有序的，可以使用二分查找算法来加快查找速度。

如果数据具有某种模式或分布特征，可以使用这些特征来优化查找策略。